NAME: SHANTHOSH S
ROLL NO: 240701487

EXPERIMENT – 2
LOW-CODE DEVELOPMENT PLATFORMS AND TRADITIONAL CODING: A COMPARATIVE STUDY

Aim

To examine the differences between low-code application development and conventional software programming in terms of workflow, effort, flexibility and practical limitations.

Introduction

Software applications can be created either by writing most of the source code manually or by using visual development platforms that provide ready-made components. This experiment compares these two methods and relates them to a Hospital Management System built with Zoho Creator.

Traditional Development Approach

In the conventional method, developers implement application features directly through programming languages and development tools. They design the data model, create user interfaces and APIs, write business rules, configure deployment and test the integrated system. This method offers extensive control over architecture and behaviour, but it generally requires experienced programmers, more development time and a larger testing effort.

Low-Code Development Approach

A low-code environment provides visual tools such as form builders, reusable components and workflow configuration. Developers assemble much of the application through drag-and-drop actions and configure the platform's built-in services. This can speed up delivery and reduce the amount of code required. However, advanced customisation may be limited by the platform, and the application can become dependent on its provider.

Example: Hospital Management System

For a Hospital Management System, a low-code platform such as Zoho Creator can be used to create patient, doctor, appointment, admission and billing modules through forms and workflows. A conventional implementation would require the team to develop the database, interfaces, APIs, application logic and deployment setup manually.

Traditional Development Lifecycle

The traditional development lifecycle proceeds through requirement analysis, architecture and design, manual implementation, quality assurance, system integration, production deployment and continuing support.

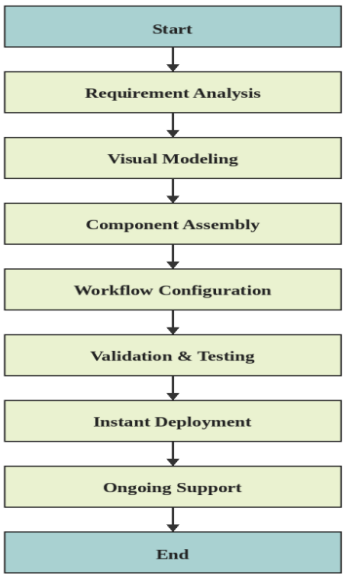


Figure 2: Low-code development lifecycle using Zoho Creator

Figure 1: Stages in the traditional software development lifecycle.

Low-Code Development Lifecycle

The low-code lifecycle begins with requirement analysis. The application is then modelled visually, assembled from platform components and configured with workflows. The solution is validated, tested, deployed and maintained through the platform.

Figure 2: Stages in a low-code development process using Zoho Creator.

Comparative Analysis

Feature	Traditional Development	Low-Code Development
Build Time	Usually longer because features are programmed and checked individually.	Often shorter because prebuilt components and visual tools accelerate assembly.
Technical Skills	Requires knowledge of programming languages, frameworks and development tools.	Visual builders make basic application creation accessible with limited coding experience.
Flexibility	Offers broad freedom to implement specialised logic and architecture.	Customisation depends on the available components, integrations and platform limits.
Cost	Higher initial costs may arise from specialist staff and infrastructure.	Lower initial effort, although subscription or licence fees may continue.
Maintenance	Updates and fixes generally involve code changes followed by testing.	The platform provider manages many underlying updates; app-specific changes may still be required.
Vendor Dependence	The development team can retain control over its source code and hosting choices.	The solution is tied to the provider's services, ecosystem and export options.
Suitable Uses	Complex, performance-critical or tightly regulated systems.	Prototypes, internal business tools and routine workflow automation.

Conclusion

Traditional programming is appropriate when a project needs extensive control, specialised behaviour or demanding performance. Low-code development is useful when the priority is to build and release standard business workflows quickly. In a Hospital Management System, forms and routine processes can be handled with low-code tools, while complex backend requirements may call for conventional development.